ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers

Gökay Gülsoy 323078045

CENG 543 Information Retrieval and Modern NLP
İzmir Institute of Technology

December 22, 2025

Table of Contents

1. Introduction
2. Related Literature & Limitations
3. Proposed Methodology
4. Experimental Results
5. Closing Statements
6. references

Issue of Practically Utilizing Large-Scale Transformers

- How to efficiently serve ever-larger trained natural language models in practice has become challenging even for powerful cloud servers due to memory and computation requirements.

Issue of Practically Utilizing Large-Scale Transformers

- How to efficiently serve ever-larger trained natural language models in practice has become challenging even for powerful cloud servers due to memory and computation requirements.
- This work present an efficient and affordable post-training quantization approach to compress large Transformer-based models, termed as ZeroQuant.

Main Components of ZeroQuant Pipeline

1. A fine-grained hardware-friendly quantization scheme for both weights and activations.
2. A novel, affordable layer-by-layer knowledge distillation algorithm (LKD) even without access to original data.
3. A highly optimized quantization system backend support to remove the quantization/de-quantization overhead.

Quantization

- Memory footprint and the computational cost to deploy them become a major bottleneck, even on the cloud severs with the powerful GPU devices.

Quantization

- Memory footprint and the computational cost to deploy them become a major bottleneck, even on the cloud severs with the powerful GPU devices.
- One promising way to alleviate this challenge is quantization, which can reduce the bit precision for both weight and activations for a lower memory footprint and faster computation.

Quantization

- Memory footprint and the computational cost to deploy them become a major bottleneck, even on the cloud severs with the powerful GPU devices.
- One promising way to alleviate this challenge is quantization, which can reduce the bit precision for both weight and activations for a lower memory footprint and faster computation.
- However, quantization usually requires retraining to recover from degradation from representation of weights and activations.

Table of Contents

1 Introduction

2 Related Literature & Limitations

3 Proposed Methodology

4 Experimental Results

5 Closing Statements

6 references

What are the studies that have already been done ?

- Recently, zero-shot quantization [1, 2], and post-training quantization(PTQ) [3, 4] are proposed to address the training-data access and compute requirement challenges since PTQ generally requires no retraining.

What are the studies that have already been done ?

- Recently, zero-shot quantization [1, 2], and post-training quantization(PTQ) [3, 4] are proposed to address the training-data access and compute requirement challenges since PTQ generally requires no retraining.
- Along this line, one of the first works applied to Transformer-based models is [5]. The authors introduced a centroid-based quantization method where outlier numbers use FP32 format and the rest of the numbers are quantized using non-uniform quantization method.

What are the studies that have already been done ?

- Recently, zero-shot quantization [1, 2], and post-training quantization(PTQ) [3, 4] are proposed to address the training-data access and compute requirement challenges since PTQ generally requires no retraining.
- Along this line, one of the first works applied to Transformer-based models is [5]. The authors introduced a centroid-based quantization method where outlier numbers use FP32 format and the rest of the numbers are quantized using non-uniform quantization method.
- More recently, [6], introduced high-precision activation quantization for part of the model to overcome the high dynamic activation ranges.

Limitations of Existing Approaches

- Even though PTQ does not require retraining, it often induces a significant drop in accuracy because the network can be sensitive to quantization errors.

Limitations of Existing Approaches

- Even though PTQ does not require retraining, it often induces a significant drop in accuracy because the network can be sensitive to quantization errors.
- Existing work often do not discuss the quantization and dequantization cost associated with different quantization schemes, which in fact has a big impact on the performance benefit of using low precision.

Limitations of Existing Approaches

- For extreme quantization, knowledge distillation is usually used to boost performance, which adds another source of expensive computation cost.

Limitations of Existing Approaches

- For extreme quantization, knowledge distillation is usually used to boost performance, which adds another source of expensive computation cost.
- To achieve better accuracy performance, hidden-state knowledge distillation is applied for a quantized model. This would put significant pressure on the GPU memory and the compute resources.

Table of Contents

1 Introduction

2 Related Literature & Limitations

3 Proposed Methodology

4 Experimental Results

5 Closing Statements

6 references

Proposed Methodology

- **Dynamic Activation Range:** Larger variance in the activation range makes it difficult to use a fixed quantization range for all tokens to retain the prediction accuracy, because the limited representation power for small range tokens hurts accuracy performance.
- **Different Ranges of Neurons in Weight Matrices:** There is a $10x$ difference between the largest magnitudes of different rows, and this leads to worse generation performance of the INT8 weight PTQ. This also makes it very challenging when INT4 quantization is applied, as $10x$ smaller range leads to 2 or 3 numbers for the representations.

Phases of Followed Methodology Cont.

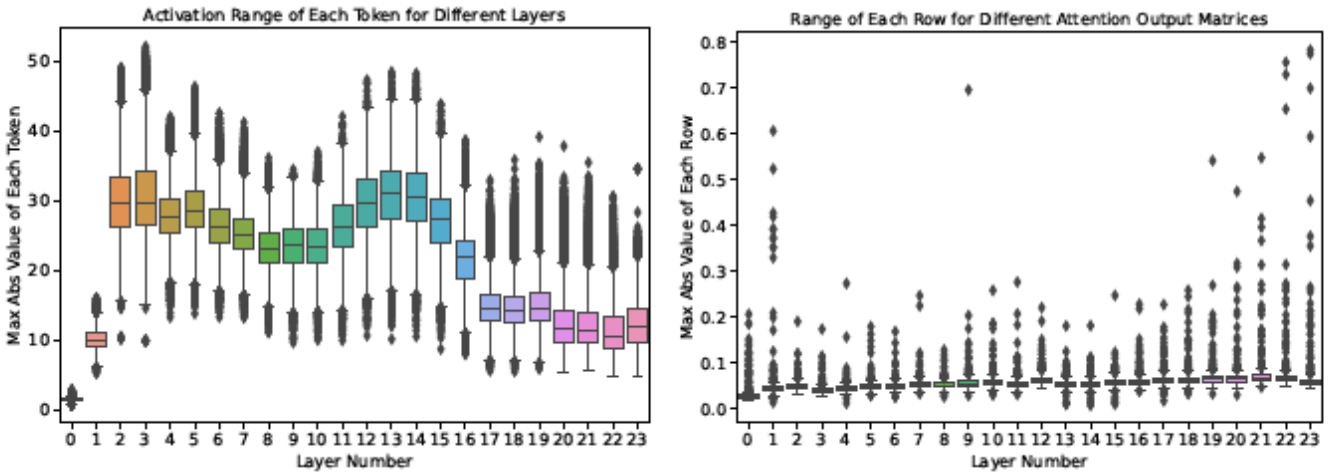

Figure: The activation range (left) and row-wise weight range of the attention output matrix (right) of different layers on the pretrained $GPT-3_{350M}$

Fine Grained Hardware-friendly Quantization Scheme

- **Group-wise Quantization for Weights:** Group-wise weight matrix quantization was first proposed in [7], where the weight matrix $W \in R^{nxm}$ is partitioned into g groups, and each group is quantized separately.

Fine Grained Hardware-friendly Quantization Scheme

- **Group-wise Quantization for Weights:** Group-wise weight matrix quantization was first proposed in [7], where the weight matrix $W \in R^{nxm}$ is partitioned into g groups, and each group is quantized separately.
- In this study, researchers considered the hardware constraint from Ampere Architecture of GPUs, where the compute unit is based on Warp Matrix Multiply and Accumulate tiling size to achieve better speedup.

Fine Grained Hardware-friendly Quantization Scheme

- **Group-wise Quantization for Weights:** Group-wise weight matrix quantization was first proposed in [7], where the weight matrix $W \in R^{nxm}$ is partitioned into g groups, and each group is quantized separately.
- In this study, researchers considered the hardware constraint from Ampere Architecture of GPUs, where the compute unit is based on Warp Matrix Multiply and Accumulate tiling size to achieve better speedup.
- This group-wise quantization leads to much better accuracy as compared to single-matrix quantization due to its finer granularity quantization while still achieving great latency reduction.

Token-wise Quantization for Activations

- Common practice for existing PTQ work is to use static quantization for activation, where the min/max range is calculated at an offline calibration phase. Such a method might be sufficient for small-scale models where the variance in the activation range is small.

Token-wise Quantization for Activations

- Common practice for existing PTQ work is to use static quantization for activation, where the min/max range is calculated at an offline calibration phase. Such a method might be sufficient for small-scale models where the variance in the activation range is small.
- Token-wise quantization can significantly reduce the representation error for quantized activations. Also, as it does not need to calibrate the activation range, there is no quantization-related cost for a moderate quantization scheme (INT8 weight with INT8 activation) for ZeroQuant.

Layer-by-layer Knowledge Distillation with Affordable Cost

- Knowledge distillation (KD) is one of the most powerful methods to alleviate the accuracy degradation after model compression.

Layer-by-layer Knowledge Distillation with Affordable Cost

- Knowledge distillation (KD) is one of the most powerful methods to alleviate the accuracy degradation after model compression.
- KD needs to hold a teacher and student model together during the training, which dramatically increases the memory and compute cost.

Layer-by-layer Knowledge Distillation with Affordable Cost

- Knowledge distillation (KD) is one of the most powerful methods to alleviate the accuracy degradation after model compression.
- KD needs to hold a teacher and student model together during the training, which dramatically increases the memory and compute cost.
- KD usually requires full training of the student model. Therefore, several copies of weight parameters need to be stored in memory to update the model.

Layer-by-layer Knowledge Distillation with Affordable Cost

- Knowledge distillation (KD) is one of the most powerful methods to alleviate the accuracy degradation after model compression.
- KD needs to hold a teacher and student model together during the training, which dramatically increases the memory and compute cost.
- KD usually requires full training of the student model. Therefore, several copies of weight parameters need to be stored in memory to update the model.
- KD generally requires original training data, which sometimes is not accessible due to privacy/confidentiality issues

Layer-by-layer Knowledge Distillation with Affordable Cost

$$\mathcal{L}_{LKD,k} = MSE(L_k \cdot L_{k-1} L_{k-2} \cdot \cdots - \hat{L}_k \cdot L_{k-1} \cdot L_{k-2}) \qquad (1)$$

- To address those limitations, researchers presented a layer-by-layer distillation (LKD) algorithm. LKD quantizes the network layer by layer and uses its original version as the teacher model. More specifically, layer L_k is going to be quantized, and its quantized version is $\hat{L}_k$.

Layer-by-layer Knowledge Distillation with Affordable Cost

Precision (Method)	CoLA	MNLI-m	MNLI-mm	MRPC	QNLI	QQP	RTE	SST-2	STS-B	Ave.	Ave. Time (s)
W16A16 (Baseline)	59.72	84.94	85.06	86.27/90.57	92.15	91.51/88.56	72.20	93.23	90.06/89.59	83.95	N/A
W8A8 [56] (QAT)$^+$	—	83.91	83.83	—	—	—	—	92.83	—	—	—
W8A8 [76] (QAT)	58.48	—	—	—/89.56	90.62	—/87.96	68.78	92.24	89.04/—	—	—
W8A8 (QAT)	61.21	84.80	84.64	83.82/88.85	91.29	91.29/88.28	71.12	92.89	88.39/88.18	83.37	2900
W8A8 (PTQ)	56.06	79.99	81.06	75.49/79.67	87.35	89.92/86.82	48.38	91.40	86.58/86.44	77.41	6
W8A8/16 [6] (PTQ)*	58.63	82.67	82.67	88.74	90.41	89.40	68.95	92.66	88.00	82.46	Unknown
W8A8 (ZeroQuant)	59.59	84.83	85.13	86.03/90.39	91.98	91.45/88.46	71.12	93.12	90.09/89.62	83.75	0
W4/8A16 (PTQ)	0.00	16.74	16.95	31.62/0.00	50.74	63.18/0.00	47.29	70.64	16.48/15.91	33.11	6
W4/8A16 (ZeroQuant)	57.29	82.69	83.27	84.56/88.40	90.04	86.52/79.49	70.76	92.78	88.46/88.61	81.65	0
W4/8A16 (ZeroQuant-LKD)	58.50	83.16	83.69	84.80/89.31	90.83	88.94/84.12	70.04	92.78	88.49/88.67	82.35	31
W4/8A8 (ZeroQuant)	56.69	82.46	83.06	84.07/88.03	90.13	87.04/80.50	70.76	92.78	88.07/88.44	81.55	0
W4/8A8 (ZeroQuant-LKD)	58.80	83.09	83.65	85.78/89.90	90.76	89.16/84.85	71.84	93.00	88.16/88.55	82.71	31

Figure: Result of $BERT_{base}$ on the development set of GLUE benchmark

Quantization-Optimized Transformer Kernels

- Both optimizing the inference latency and model size is crucial for serving large-scale transformer models in practice.

Quantization-Optimized Transformer Kernels

- Both optimizing the inference latency and model size is crucial for serving large-scale transformer models in practice.
- During inference, the batch size is often relatively small, so the inference latency of the model depends primarily on the time to load the inference data from the main memory.

Quantization-Optimized Transformer Kernels

- Both optimizing the inference latency and model size is crucial for serving large-scale transformer models in practice.
- During inference, the batch size is often relatively small, so the inference latency of the model depends primarily on the time to load the inference data from the main memory.
- By quantizing the weights and activations to lower precision, the data volume needed to load those data is reduced.

Quantization-Optimized Transformer Kernels Cont'd

- **CUTLASS INT8 GeMM:** researchers used CUTLASS INT8 GeMM implementation tuned for different batch sizes. Unlike standard GPU backend libraries, such as cuDNN, using CUTLASS allows more flexible quantization to reduce kernel launching and data-movement overhead.

[1] CUTLASS Link: https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda

Quantization-Optimized Transformer Kernels Cont'd

- **CUTLASS INT8 GeMM:** researchers used CUTLASS INT8 GeMM implementation tuned for different batch sizes. Unlike standard GPU backend libraries, such as cuDNN, using CUTLASS allows more flexible quantization to reduce kernel launching and data-movement overhead.
- **Fusing Token-wise Activation Quantization:** Token-wise quantization/dequantization introduces many additional operations that lead to extra data movement cost. To eliminate these costs, researchers used kernel fusion [8] to fuse quantization with previous operations such as bias-add, GeLU, and LayerNorm into a single operator.

[1] CUTLASS Link:
https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda

Quantization-Optimized Transformer Kernels Cont'd

Figure: The illustration of normal (left) and fused (right) INT8 GeMM

- Researchers presented those optimizations that maximize the memory bandwidth utilization to speed up inference latency for ZeroQuant.

Table of Contents

1 Introduction

2 Related Literature & Limitations

3 Proposed Methodology

4 Experimental Results

5 Closing Statements

6 references

Results

- To evaluate the proposed ZeroQuant, researchers tested it on both the BERT and GPT-3 models.
- For BERT, they tested $BERT_{base}$ and $BERT_{large}$ on GLUE benchmark.
- For GPT-3-style models, they tested $GPT-3_{350M}$ and $GPT-3_{1.3B}$ on 20 zero-shot evaluation tasks, including 19 accuracy-based tasks and 1 language modeling generation task.
- **Notation Explanation:** W_xA_y represents x-bit for weight quantization and y-bit for activation quantization.

Results of $BERT_{large}$

Precision (Method)	CoLA	MNLI-m	MNLI-mm	MRPC	QNLI	QQP	RTE	SST-2	STS-B	Ave.	Ave. Time (s)
W16A16 (Baseline)	63.35	86.65	85.91	87.99/91.62	92.24	91.08/88.08	74.01	93.46	90.34/90.11	85.03	N/A
W8A8 [76] (QAT)	—	—	—	—/90.9	91.74				90.12/—	—	—
W8A8 (QAT)$^{+}$	59.85	86.65	86.35	85.29/89.43	92.55	91.60/88.60	61.37	93.23	87.55/87.65	82.78	7181
W8A8 (PTQ)	60.57	75.69	76.94	81.13/84.93	88.49	84.04/74.35	46.93	91.74	62.75/55.77	73.54	31
W8A8 (ZeroQuant)	63.38	86.52	85.64	87.75/91.50	92.31	91.09/88.05	72.56	93.35	90.45/90.19	84.81	0
W4/8A16 (PTQ)	0.00	16.85	33.24	68.38/80.89	51.25	63.18/0.00	52.71	52.41	-5.74/-8.51	35.73	31
W4/8A16 (ZeroQuant)	62.99	84.77	84.42	87.50/91.16	91.63	90.03/86.41	48.01	92.16	89.49/89.28	81.23	0
W4/8A16 (ZeroQuant-LKD)	63.72	84.90	84.81	87.99/91.39	91.45	90.34/86.92	51.62	92.43	89.46/89.29	81.85	550
W4/8A8 (ZeroQuant)	62.34	84.62	84.25	87.75/91.38	91.87	89.86/86.09	47.65	91.97	89.39/89.17	81.06	0
W4/8A8 (ZeroQuant-LKD)	63.51	84.70	84.71	88.73/91.99	91.73	90.25/86.74	49.82	92.09	89.34/89.08	81.62	550

Figure: Result of $BERT_{large}$ set of GLUE benchmark

Results of $GPT3 - 3_{350M}$

Precision (Method)	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)	Time Cost
W16A16	49.3	66.3	29.4	53.8	75.1	38.9	21.5	N/A
W8A8 (PTQ)	42.6	64.1	28.0	53.1	67.5	37.8	26.2	7 mins
W8A8 (ZeroQuant)	51.0	66.5	29.2	53.4	74.9	38.7	21.7	0
W4/8A16 (PTQ)	0.00	51.4	30.2	52.7	16.1	28.9	1.76e5	7 mins
W4/8A16 (ZeroQuant)	10.1	58.5	27.2	52.0	56.5	33.5	88.6	0
W4/8A16 (ZeroQuant-LKD)	39.8	63.8	29.4	53.1	70.1	37.0	30.6	1.1 hours
W4/8A8 (ZeroQuant)	10.5	57.7	28.0	52.7	55.3	33.4	92.1	0
W4/8A8 (ZeroQuant-LKD)	37.4	61.8	28.2	53.1	68.5	36.6	31.1	1.1 hours

Figure: Results of $GPT3 - 3_{350M}$ on 20 zero-shot evaluation datasets

Results of $GPT3 - 3_{1.3B}$

Precision (Method)	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)	Time Cost
W16A16	49.3	66.3	29.4	53.8	75.1	38.9	21.5	N/A
W8A8 (PTQ)	42.6	64.1	28.0	53.1	67.5	37.8	26.2	7 mins
W8A8 (ZeroQuant)	51.0	66.5	29.2	53.4	74.9	38.7	21.7	0
W4/8A16 (PTQ)	0.00	51.4	30.2	52.7	16.1	28.9	1.76e5	7 mins
W4/8A16 (ZeroQuant)	10.1	58.5	27.2	52.0	56.5	33.5	88.6	0
W4/8A16 (ZeroQuant-LKD)	39.8	63.8	29.4	53.1	70.1	37.0	30.6	1.1 hours
W4/8A8 (ZeroQuant)	10.5	57.7	28.0	52.7	55.3	33.4	92.1	0
W4/8A8 (ZeroQuant-LKD)	37.4	61.8	28.2	53.1	68.5	36.6	31.1	1.1 hours

Figure: Results of $GPT3 - 3_{1.3B}$ on 20 zero-shot evaluation datasets

BERT Model Latencies

Seq Len	Precision	128								256							
BS		1	2	4	8	16	16	64	128	1	2	4	8	16	16	64	128
$BERT_{base}$	W16A16	2.45	3.22	3.85	5.51	9.96	17.93	34.25	67.08	3.13	4.05	5.70	10.55	19.27	36.69	71.75	140.0
	W8A8	1.08	1.16	1.42	1.76	2.58	3.90	6.74	12.92	1.22	1.44	2.08	2.88	4.10	7.80	14.66	28.13
	Speedup	2.27	2.78	2.71	3.13	3.86	4.60	5.08	5.19	2.57	2.81	2.74	3.66	4.70	4.70	4.89	4.98
$BERT_{large}$	W16A16	5.45	6.38	8.73	13.88	26.34	48.59	92.49	183.4	6.39	8.94	14.66	27.99	51.94	98.78	195.9	384.5
	W8A8	2.08	2.58	2.84	3.79	6.21	10.28	18.86	36.62	2.55	3.36	4.16	6.88	11.61	21.20	41.24	79.90
	Speedup	2.62	2.47	3.07	3.66	4.24	4.73	4.90	5.01	2.51	2.66	3.52	4.07	4.47	4.66	4.75	4.81

Figure: End-to-end average latency for the entire BERT model, and the time reported in milliseconds

Ablation Studies on $BERT_{large}$

GQ	TQ	LKD	CoLA	MNLI-m	MNLI-mm	MRPC	QNLI	QQP	RTE	SST-2	STS-B	Ave.
✗	✗	✗	-0.79	33.07	32.94	68.38/80.54	49.42	63.18/0.00	52.71	52.29	-4.27/-1.90	35.85
✓	✗	✗	59.81	66.63	68.79	68.63/71.17	83.87	78.24/61.30	46.93	89.45	54.58/32.52	66.52
✓	✓	✗	62.34	84.62	84.25	87.75/91.38	91.87	89.86/86.09	47.65	91.97	89.39/89.17	81.06
✓	✓	✓	63.51	84.70	84.71	88.73/91.99	91.73	90.25/86.74	49.82	92.09	89.34/89.08	81.62

Figure: Ablation study of different components for $BERT_{large}$ on the development set of GLUE. Here, GP is the abbreviation of group-wise weight quantization, and TQ is the abbreviation of token-wise activation quantization.

Quantization Technique Used in Study

- Quantization maps high-precision numbers, e.g., FP16/FP32, to their low-precision counterpart, e.g., INT4/INT8, to reduce the model footprint and improve the compute performance.
- In this work, researchers used uniform symmetric scalar quantizers. That is to say, if we have a vector/matrix, x, the quantization is applied as

$$x_{quantize} = round\left(clamp(\frac{x}{S}, -2^{bit-1}, 2^{bit-1} - 1)\right) \tag{2}$$

- where *bit* is the number of bits used to represent the quantized value, and S is the scaling factor. For weight matrix quantization, S is generally computed as $S = max(abs(x))$, since the weight matrix is static during inference.

$GPT - 3_{350M}$ Accuracy Results

Tasks	Baseline	PTQ				ZeroQuant			ZeroQuant-LKD	
	W32A32	W8A32	W32A8	W8A8	W4/8A32	W8A8	W4/8A32	W4/8A8	W4/8A32	W4/8A8
HellaSwag	38.6	38.1	37.6	36.8	26.5	38.4	30.4	30.5	35.3	35.3
LAMBADA	49.3	49.3	44.7	42.9	0	51.0	10.1	10.5	39.8	37.4
TriviaQA	3.00	2.67	2.70	2.32	0	2.86	0.159	0.194	1.043	0.23
WebQs	1.43	0.935	1.23	0.689	0	1.378	0.246	0.394	0.591	0.049
Winogrande	53.2	52.1	52.1	52.1	47.8	51.4	52.6	50.7	51.6	51.8
PIQA	66.3	66.1	64.8	64.1	51.4	66.5	58.5	57.7	63.8	61.8
ARC (Challenge)	24.2	24.0	24.0	24.1	27.0	24.5	22.0	21.8	21.8	23.6
ARC (Easy)	45.5	44.7	44.2	43.9	25.1	44.5	37.6	37.5	40.5	40.5
ANLI R1	31.1	30.0	31.3	33.2	33.4	31.1	32.8	32.7	32.4	33.8
ANLI R2	34.3	36.0	36.5	35.9	33.4	34.3	34.7	34.2	34.1	33.5
ANLI R3	34.1	34.0	33.0	37.2	33.5	33.4	34.9	34.5	33.1	33.4
OpenBookQA	29.4	29.6	28.2	28.0	30.2	29.2	27.2	28.0	29.4	28.2
RACE-h	32.4	31.3	30.3	30.7	22.4	32.2	25.7	26.4	29.5	29.7
BoolQ	60.3	60.2	57.0	56.9	37.8	60.2	60.1	59.4	61.9	61.9
Copa	69.0	67.0	71.0	73.0	48.0	69.0	63.0	64.0	68.0	66.0
RTE	53.8	54.2	52.7	53.1	52.7	53.4	52.0	52.7	53.1	53.1
WSC	36.5	36.5	36.5	35.6	63.5	36.5	36.5	36.5	36.5	36.5
MultiRC	0.839	0.839	0.839	0.944	0.315	0.839	1.889	1.889	0.839	0.839
ReCoRD	75.1	74.8	69.2	67.5	16.1	74.9	56.5	55.3	70.1	68.5
Wikitext-2	21.52	22.09	24.56	26.20	1.76e5	21.68	88.64	92.10	30.56	31.13
Average Acc	38.86	38.54	37.78	37.84	28.9	38.71	33.52	33.42	37.02	36.64

Figure: The full results of $GPT - 3_{350M}$

$GPT - 3_{1.3B}$ Accuracy Results

Tasks	Baseline	PTQ		ZeroQuant			ZeroQuant-LKD	
	W32A32	W8A8	W4/8A32	W8A8	W4/8A32	W4/8A8	W4/8A32	W4/8A8
HellaSwag	51.4	47.0	26.1	50.8	43.7	43.2	48.5	46.7
LAMBADA	61.3	54.8	0	62.6	43.9	46.8	59.4	48.7
TriviaQA	7.37	4.43	0	6.67	2.36	2.09	4.28	2.99
WebQs	2.90	1.476	0	2.07	1.132	1.28	1.673	1.083
Winogrande	57.1	55.7	50.1	57.1	54.6	54.3	55.3	53.8
PIQA	71.4	67.7	50.4	70.7	66.5	66.4	69.5	68.1
ARC (Challenge)	27.2	27.1	26.5	26.8	25.7	25.3	27.8	26.5
ARC (Easy)	54.5	49.7	26.0	53.8	48.0	47.0	52.2	50.3
ANLI R1	32.0	33.1	33.0	33.4	33.8	33.6	34.2	33.8
ANLI R2	32.0	32.9	33.3	33.9	33.0	33.0	33.8	32.8
ANLI R3	33.8	33.5	32.3	34.8	33.6	33.5	33.7	33.0
OpenBookQA	33.6	32.6	27.0	33.4	30.0	28.8	31.6	29.0
RACE-h	33.6	32.6	22.4	32.7	30.9	29.9	32.7	33.2
BoolQ	62.4	59.2	37.8	61.3	60.3	59.8	61.7	61.3
Copa	70.0	70.0	55.0	72.0	73.0	74.0	72.0	70.0
RTE	53.1	54.5	50.9	52.7	52.7	52.7	52.7	52.0
WSC	37.5	36.5	63.5	36.5	36.5	36.5	36.5	36.5
MultiRC	1.05	0.839	0.315	0.839	1.259	1.154	0.839	0.839
ReCoRD	82.6	75.7	15.8	80.9	77.3	76.2	79.7	77.4
Wikitext-2	15.3	18.85	1.35e5	15.69	21.9	24.09	17.56	18.18
Average Acc	42.36	40.49	28.97	42.26	39.38	39.24	41.48	39.90

Figure: The full results of $GPT - 3_{1.3B}$

$GPT - 3_{350M}$ Results Using Different Data Sources

Tasks	Random Data	Wikipedia	Original Training Data
HellaSwag	33.9	35.5	35.3
LAMBADA	26.1	33.9	37.4
TriviaQA	0.088	0.972	0.23
WebQs	0.049	0.344	0.049
Winogrande	50.3	52.4	51.8
PIQA	59.3	62.4	61.8
ARC (Challenge)	22.6	23.3	23.6
ARC (Easy)	38.3	40.0	40.5
ANLI R1	33.0	32.0	33.8
ANLI R2	34.3	34.7	33.5
ANLI R3	33.4	32.9	33.4
OpenBookQA	29.2	28.0	28.2
RACE-h	27.8	29.1	29.7
BoolQ	47.8	52.6	61.9
Copa	65.0	69.0	66.0
RTE	50.5	52.7	53.1
WSC	36.5	36.5	36.5
MultiRC	1.574	1.154	0.839
ReCoRD	64.9	69.5	68.5
Wikitext-2	40.63	30.36	31.13
Average Acc	34.45	36.16	36.64

Figure: The full results of $W4/8A8$ $GPT - 3_{350M}$ using different data sources

$GPT - NeoX_{20B}$ Accuracy Results

Tasks	W16A16	W8A8/16
HellaSwag	71.4	71.2
LAMBADA	71.7	71.9
TriviaQA	25.8	25.9
WebQs	6.3	6.64
Winogrande	66.0	65.7
PIQA	77.7	78.3
ARC (Challenge)	41.0	42.2
ARC (Easy)	68.5	68.8
ANLI R1	33.1	33.9
ANLI R2	33.4	34.4
ANLI R3	35.1	35.4
OpenBookQA	39.8	38.8
RACE-h	38.5	37.6
BoolQ	69.4	69.9
Copa	84.0	85.0
RTE	54.9	54.9
WSC	50.0	44.2
MultiRC	3.57	4.41
ReCoRD	88.3	88.0
Average Acc	50.45	50.38

Figure: The full results of $GPT - NeoX_{20B}$

Table of Contents

1 Introduction

2 Related Literature & Limitations

3 Proposed Methodology

4 Experimental Results

5 Closing Statements

6 references

Questions ?

[1] Image taken from: https://business.tutsplus.com/tutorials/thank-you-slide-powerpoint-presentation--cms-34231

Table of Contents

1 Introduction

2 Related Literature & Limitations

3 Proposed Methodology

4 Experimental Results

5 Closing Statements

6 references

References I

Y. Cai, Z. Yao, Z. Dong, A. Gholami, M. W. Mahoney, and K. Keutzer, "Zeroq: A novel zero shot quantization framework," 2020.

M. Nagel, M. van Baalen, T. Blankevoort, and M. Welling, "Data-free quantization through weight equalization and bias correction," 2019.

References II

Z. Liu, Y. Wang, K. Han, W. Zhang, S. Ma, and W. Gao, "Post-training quantization for vision transformer," in *Advances in Neural Information Processing Systems* (M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, eds.), vol. 34, pp. 28092–28103, Curran Associates, Inc., 2021.

M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, and T. Blankevoort, "Up or down? adaptive rounding for post-training quantization," 2020.

References III

- A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 811–824, IEEE, 2020.
- Y. Bondarenko, M. Nagel, and T. Blankevoort, "Understanding and overcoming the challenges of efficient transformer quantization," 2021.
- S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "Q-bert: Hessian based ultra low precision quantization of bert," 2019.

References IV

G. Wang, Y. Lin, and W. Yi, "Kernel fusion: An effective method for better power efficiency on multithreaded gpu," in *Proceedings of the 2010 IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing*, GREENCOM-CPSCOM '10, (USA), p. 344–350, IEEE Computer Society, 2010.